2.4. Основы технологии работы с объектами базы данных и с данными инструментальными средствами и средствами языка SQL

2.4.1. Создание SQL-запросов

(Практическая работа №4)

Дополнительные источники:

1. Астахова И.Ф. SQL в примерах и задачах. Учеб. Пособие.Мн.: Новое знание, 2002
2. Онлайн тренажер с упражнениями по SQL. [Электронный ресурс] Режим доступа: https://sql-academy.org/ru
3. Руководство по MS SQL Server. [Электронный ресурс] Режим доступа: https://metanit.com/sql/sqlserver/

Основы Transact SQL: Простые (однотабличные) выборки данных

SQL — это аббревиатура выражения Structured Query Language (язык структурированных запросов). SQL основывается на реляционной алгебре и специально разработан для взаимодействия с реляционными базами данных.

Язык SQL представляет собой совокупность операторов, которые можно разделить на четыре группы:

- DDL (Data Definition Language) - операторы определения данных
- DML (Data Manipulation Language) - операторы манипуляции данными
- DCL (Data Control Language) - операторы определения доступа к данным
- TCL (Transaction Control Language) - операторы управления транзакциями

SQL является стандартизированным языком. Стандартный SQL поддерживается комитетом стандартов ANSI (Американский национальный институт стандартов), и соответственно называется ANSI SQL.

Многие разработчики СУБД расширили возможности SQL, введя в язык дополнительные операторы или инструкции. Эти расширения необходимы для выполнения дополнительных функций или для упрощения выполнения определенных операций. И хотя часто они очень полезны, эти расширения привязаны к определенной СУБД и редко поддерживаются более чем одним разработчиком. Все крупные СУБД и даже те, у которых есть собственные расширения, поддерживают ANSI SQL (в большей или меньшей степени). Отдельные же реализации носят собственные имена (PL-SQL, Transact-SQL и т.д.). Transact-SQL (T-SQL) – реализация языка SQL корпорации Microsoft, используемая, в частности, и в SQL Server.

Запросы на выборку данных (оператор SELECT)

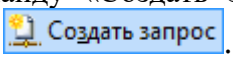
SELECT – наиболее часто используемый SQL оператор. Он предназначен для выборки информации из таблиц. Чтобы при помощи оператора SELECT извлечь данные из таблицы, нужно указать как минимум две вещи — что вы хотите выбрать и откуда.

Выборка отдельных столбцов

```
SELECT [Description]
FROM Product
```

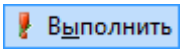
В приведенном выше операторе используется оператор SELECT для выборки одного столбца под названием Description из таблицы Product. Искомое имя столбца указывается сразу после ключевого слова SELECT, а ключевое слово FROM указывает на имя таблицы, из которой выбираются данные.

Для создания и тестирования данного запроса в Management Studio выполните следующие шаги:

1. В контекстном меню базы Sales выберите команду «Создать запрос» или щелкните соответствующую кнопку на панели инструментов 

.

2. В открывшемся окне создания нового запроса введите представленные выше инструкции SQL.

3. Для запуска запроса на выполнение щелкните кнопку 

на панели инструментов или нажмите клавишу F5. В нижней части экрана должны появиться результаты.

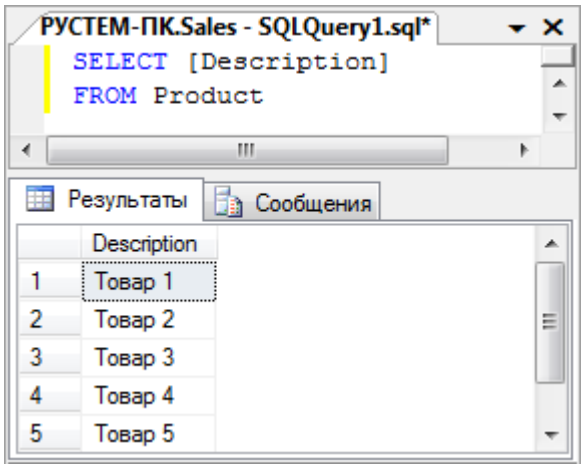


Рисунок 2.4.1.1

4. Management Studio позволяет сохранять пакеты SQL. Это полезно для сохранения сложных запросов, которые будут повторно запускаться в будущем. Для этого щелкните кнопку  на панели инструментов. По умолчанию файлы запросов сохраняются с расширением .sql. В дальнейшем сохраненный запрос может быть открыт командой «Открыть файл».

Выборка нескольких столбцов

Для выборки из таблицы нескольких столбцов используется тот же оператор SELECT. Отличие состоит в том, что после ключевого слова SELECT необходимо через запятую указать несколько имен столбцов.

```
SELECT [Description], InStock
FROM Product
```

Выборка всех столбцов

Помимо возможности осуществлять выборку определенных столбцов (одного или нескольких), при помощи оператора SELECT можно запросить все столбцы, не перечисляя каждый из них. Для этого вместо имен столбцов вставляется групповой символ “звездочка” (*). Это делается следующим образом.

```
SELECT *
FROM Product
```

Сортировка данных

В результате выполнения запроса на выборку данные выводятся в том порядке, в котором они находятся в таблице. Для точной сортировки выбранных при помощи оператора SELECT данных используется предложение ORDER BY. В этом предложении указывается имя одного или нескольких столбцов, по которым необходимо отсортировать результаты. Взгляните на следующий пример.

```
SELECT IdProd, [Description], InStock
FROM Product
ORDER BY InStock
```

Это выражение идентично предыдущему, за исключением предложения ORDER BY, которое указывает СУБД отсортировать данные по возрастанию значений столбца InStock.

Сортировка по нескольким столбцам

Чтобы осуществить сортировку по нескольким столбцам, просто укажите их имена через запятую. В следующем коде выбираются три столбца, а результат сортируется по двум из них — сначала по количеству, а потом по названию.

```
SELECT IdProd, [Description], InStock
FROM Product
ORDER BY InStock, [Description]
```

Важно понимать, что при сортировке по нескольким столбцам порядок сортировки будет таким, который указан в запросе. Другими словами, в примере, приведенном выше, продукция сортируется по столбцу Description, только если существует несколько строк с одинаковыми значениями InStock. Если никакие значения столбца InStock не совпадают, данные по столбцу Description сортироваться не будут.

Указание направления сортировки

В предложении ORDER BY можно также использовать порядок сортировки по убыванию. Для этого необходимо указать ключевое слово DESC. В следующем примере продукция сортируется по количеству в убывающем порядке плюс по названию продукта.

```
SELECT IdProd, [Description], InStock
FROM Product
ORDER BY InStock DESC, [Description]
```

Ключевое слово DESC применяется только к тому столбцу, после которого оно указано. В предыдущем примере ключевое слово DESC было указано для столбца InStock, но не для Description. Таким образом, столбец InStock отсортирован в порядке убывания, а столбец Description в возрастающем порядке (принятым по умолчанию).

Фильтрация данных

В таблицах баз данных обычно содержится много информации и довольно редко возникает необходимость выбирать все строки таблицы. Гораздо чаще бывает нужно извлечь какую-то часть данных таблицы для каких-либо действий или отчетов. Выборка только необходимых данных включает в себя критерий поиска, также известный под названием предложение фильтрации. В операторе SELECT данные фильтруются путем указания критерия поиска в предложении WHERE. Предложение WHERE указывается сразу после названия таблицы (предложения FROM) следующим образом:

```
SELECT IdProd, [Description], InStock
FROM Product
WHERE InStock = 0
```

Этот оператор извлекает значения всех столбцов из таблицы товаров, но показывает не все строки, а только те, значение в столбце InStock (Количество товаров на складе) которых равно 0, т.е. только список отсутствующих на складе товаров.

При совместном использовании предложений ORDER BY и WHERE, предложение ORDER BY должно следовать после WHERE.

В предыдущем примере проводилась проверка на равенство, т.е. определялось, содержится ли в столбце указанное значение. SQL поддерживает весь спектр условных (логических) операций, которые приведены в следующей таблице 2.4.1.1.

Таблица 2.4.1.1

Операция	Описание
=	Равенство
<>	Неравенство
!=	Неравенство
<	Меньше
<=	Меньше или равно
!<	Не меньше
>	Больше
>=	Больше или равно
!>	Не больше
BETWEEN	Между двумя указанными значениями
IS NULL	Значение NULL

В следующем примере осуществляется выборка всех клиентов, для которых не указан контактный телефон.

```
SELECT FName, LName, Phone
FROM Customer
WHERE PHONE IS NULL
```

Для поиска диапазона значений можно использовать операцию BETWEEN. Ее синтаксис немного отличается от других операций предложения WHERE, так как для нее требуются два значения: начальное и конечное. Например, операцию BETWEEN можно использовать для поиска товаров, количество которых находится в промежутке между 5 и 10.

```
SELECT IdProd, [Description], InStock
FROM Product
WHERE InStock BETWEEN 5 AND 10
```

Для объединения в предложении WHERE нескольких условий необходимо использовать логические операторы AND и (или) OR. Оператор AND требует одновременного выполнения обоих условий. Запишем предыдущий запрос посредством объединения двух операции сравнения оператором AND.

```
SELECT IdProd, [Description], InStock
FROM Product
WHERE (InStock >= 5) AND (InStock <= 10)
```

Ключевое слово AND указывает СУБД возвращать только те строки, которые удовлетворяют всем перечисленным критериям отбора. В данном случае будут выбраны только те товары, количество которых находится в промежутке от 5 до 10.

Оператор OR указывает СУБД выбирать только те строки, которые удовлетворяют хотя бы одному из условий.

```
SELECT IdCity, CityName
FROM City
WHERE (CityName = 'Москва') OR (CityName = 'Казань')
```

Посредством этого SQL запроса из справочника городов выбираются только Москва и Казань. Ключевое слово OR указывает СУБД использовать какое-то одно условие, а не сразу два. Если бы здесь использовалось ключевое слово AND, мы бы не получили никаких данных.

Если вы внимательно рассмотрите выражение в предыдущем предложении WHERE, то заметите, что значения, с которыми сравниваются названия городов, заключены в одинарные кавычки. Одинарные кавычки используются для определения границ строки (строковой константы). При работе со строковыми константами их всегда необходимо отделять одинарными кавычками.

Предложения WHERE могут содержать любое количество логических операторов AND и OR. Комбинируя их можно создавать сложные фильтры. Однако при комбинировании ключевых слов AND и OR необходимо учитывать, что оператор AND выполняется раньше оператора OR, т.е. имеет более высокий приоритет. Изменить приоритет можно с помощью круглых скобок.

В следующем примере осуществляется выборка из таблицы клиентов всех Ивановых и Петровых, для которых не указан контактный телефон.

```
SELECT FName, LName, Phone
FROM Customer
WHERE (LName = 'Иванов' OR LName = 'Петров') AND PHONE IS NULL
```

В случае отсутствия скобок результат был бы не верным, а именно включал бы в себя всех Петровых без контактного телефона и всех Ивановых без каких либо ограничений.

Для определения входит ли сравниваемое значение в определенное заданное множество можно воспользоваться оператором IN. При этом все допустимые значения, заключенные в скобки, перечисляются через запятую. В частности предыдущий пример с использованием оператора IN может быть записан в более компактной форме.

```
SELECT FName, LName, Phone
FROM Customer
WHERE LName IN ('Иванов','Петров') AND PHONE IS NULL
```

Для отрицания какого-то условия используется логический оператор NOT. Поскольку NOT никогда не используется сам по себе (а только вместе с другими логическими операторами), его синтаксис немного отличается от синтаксиса остальных операторов. В отличие от них, NOT вставляется перед названием столбца, значения которого нужно отфильтровать, а не после. В следующем примере отбираются все клиенты, для которых имеются сведения об их контактом телефоне.

```
SELECT FName, LName, Phone
FROM Customer
WHERE NOT PHONE IS NULL
```

Для фильтрации данных по критерию соответствия определенной символьной строки заданному шаблону используется оператор LIKE. Шаблон может включать обычные символы и символы-шаблоны. Во время сравнения с шаблоном необходимо, чтобы его обычные символы в точности совпадали с символами, указанными в строке. Символы-шаблоны могут совпадать с произвольными элементами символьной строки. Использование символов-шаблонов с оператором LIKE предоставляет больше возможностей, чем использование обычных операторов сравнения. Шаблон может включать в себя следующие символы-шаблоны (см. таблицу 2.4.1.2).

Таблица 2.4.1.2

Символ-шаблон	Описание	Пример
%	Любое количество символов	Инструкция WHERE FName LIKE 'А%' выполняет поиск и выдает всех клиентов, имена которых начинаются на букву ‘А’.
_	Любой одиночный символ	Инструкция WHERE LName LIKE '_етров' выполняет поиск и выдает всех клиентов, фамилии которых состоят из шести букв и заканчиваются сочетанием ‘етров’ (Петров, Ветров и т.п.).
[]	Любой символ, указанный в квадратных скобках	Инструкция WHERE LName LIKE '[Л-С]омов' выполняет поиск и выдает всех клиентов, фамилии которых заканчиваются на ‘омов’ и начинаются на любую букву в промежутке от ‘Л’ до ‘С’, например Ломов, Ромов, Сомов и т.п.
[^]	Любой символ, кроме перечисленных в квадратных скобках	Инструкция WHERE LName LIKE 'ив[^а]%' выполняет поиск и выдает всех клиентов, фамилии которых начинаются на ‘ив’ и третья буква отличается от ‘а’.

В следующем примере осуществляется выборка всех товаров, названия которых начинаются на букву Т.

```
SELECT *
FROM Product
WHERE [Description] LIKE 'Т%'
```

Создание вычисляемых полей

Конструкция SELECT кроме имен столбцов таблиц может также включать так называемые вычисляемые поля. В отличие от всех выбранных нами ранее столбцов, вычисляемых полей на самом деле в таблицах базы данных нет. Они создаются "на лету" SQL-оператором SELECT. Рассмотрим следующий пример.

```
SELECT IdCust AS 'Номер клиента', FName + ' ' +LName AS 'Фамилия и имя клиента'
FROM Customer
```

Здесь создается вычисляемое поле, которому с помощью ключевого слова AS дан псевдоним ‘Фамилия и имя клиента’. Оно позволяет объединить (произвести конкатенацию) с помощью оператора + фамилию, пробел и имя клиента в одно поле (столбец). Псевдоним может быть задан и для обычного столбца таблицы. В частности здесь столбцу IdCust задан псевдоним ‘Номер клиента’.

Еще одним способом использования вычисляемых полей является выполнение математических операций над выбранными данными. Рассмотрим пример.

```
SELECT IdProd, Qty, Price, Qty * Price AS 'Стоимость'
FROM OrdItem
WHERE IdOrd = 1
```

Здесь с помощью оператора умножения * вычисляется общая стоимость каждого товара в заказе с кодом 1 как произведение количества на цену.

Исключение дублирующих записей

Для исключения из результата выборки повторяющихся строк используется ключевое слово DISTINCT, которое указывается сразу после SELECT. В следующем примере осуществляется вывод всех фамилий клиентов. Даже если среди них есть однофамильцы, каждая фамилия будет выведена только один раз.

```
SELECT DISTINCT LName
FROM Customer
```

Задание для самостоятельной работы: Сформулируйте на языке SQL запросы на выборку следующих данных:

– Список всех заказов за определенный период времени (например, сентябрь 2010 года) отсортированный по дате заказа;

– Список всех товаров, названия которых включают слово ‘монитор’ с указанием их остатка на складе.

Использование агрегатных функций

В SQL определено множество встроенных функций различных категорий, среди которых особое место занимают агрегатные функции, оперирующие значениями столбцов множества строк и возвращающие одно значение. Аргументами агрегатных функций могут быть как столбцы таблиц, так и результаты выражений над ними. Агрегатные функции и сами могут включаться в другие арифметические выражения. В следующей таблице приведены наиболее часто используемые стандартные унарные агрегатные функции (см. таблицу 2.4.1.3).

Таблица 2.4.1.3

Функция	**Возвращаемое значение**
COUNT	Количество значений в столбце или строк в таблице
SUM	Сумма
AVG	Среднее
MIN	Минимум
MAX	Максимум

Общий формат унарной агрегатной функции следующий:

имя_функции([ALL | DISTINCT] выражение)

где DISTINCT указывает, что функция должна рассматривать только различные значения аргумента, а ALL — все значения, включая повторяющиеся (этот вариант используется по умолчанию). Например, функция AVG с ключевым словом DISTINCT для строк столбца со значениями 1, 1, 1 и 3 вернет 2, а при наличии ключевого слова ALL вернет 1,5.

Агрегатные функции применяются во фразах SELECT и HAVING. Здесь мы рассмотрим их использование во фразе SELECT. В этом случае выражение в аргументе функции применяется ко всем строкам входной таблицы фразы SELECT. Кроме того, во фразе SELECT нельзя использовать и агрегатные функции, и столбцы таблицы (или выражения с ними) при отсутствии фразы GROUP BY, которую мы рассмотрим в следующем разделе.

Функция COUNT имеет два формата. В первом случае возвращается количество строк входной таблицы, во втором случае — количество значений аргумента во входной таблице:

- COUNT(*)
- COUNT([DISTINCT | ALL] выражение)

Простейший способ использования этой функции — подсчет количества строк в таблице (всех или удовлетворяющих указанному условию). Для этого используется первый вариант синтаксиса.

Запрос: Количество видов продукции, информация о которых имеется в базе данных.

```
SELECT COUNT(*) AS 'Количество видов продукции'
FROM Product
```

Во втором варианте синтаксиса функции COUNT в качестве аргумента может быть использовано имя отдельного столбца. В этом случае подсчитывается количество либо всех значений в этом столбце входной таблицы, либо только неповторяющихся (при использовании ключевого слова DISTINCT).

Запрос: Количество различных имен, содержащихся в таблице Customer.

```
SELECT COUNT(DISTINCT FNAME)
FROM Customer
```

Использование остальных унарных агрегатных функции аналогично COUNT за тем исключением, что для функций MIN и MAX использование ключевых слов DISTINCT и ALL не имеет смысла. С функциями COUNT, MAX и MIN кроме числовых могут использоваться и символьные поля. Если аргумент агрегатной функции не содержит значений, функция COUNT возвращает 0, а все остальные - значение NULL.

Запрос: Дата последнего заказа до 1 сентября 2010 года.

```
SELECT MAX(OrdDate)
FROM [Order]
WHERE OrdDate<'1.09.2010'
```

Задание для самостоятельной работы: Сформулируйте на языке SQL запросы на выборку следующих данных:

- Суммарная стоимость всех заказов;
- Количество различных городов, содержащихся в таблице Customer.

Запросы с группировкой строк

Описанные выше агрегатные функции применялись ко всей таблице. Однако часто при создании отчетов появляется необходимость в формировании промежуточных итоговых значений, то есть относящихся к данным не всей таблицы, а ее частей. Для этого предназначена фраза GROUP BY. Она позволяет все множество строк таблицы разделить на группы по признаку равенства значений одного или нескольких столбцов (и выражений над ними). Фраза GROUP BY должна располагаться вслед за фразой WHERE (если она отсутствует, то за фразой FROM).

При наличии фразы GROUP BY фраза SELECT применяется к каждой группе, сформированной фразой группировки. В этом случае и действие агрегатных функций, указанных во фразе SELECT, будет распространяться не на всю результирующую таблицу, а только на строки в пределах каждой группы. Каждое выражение в списке фразы SELECT должно принимать единственное значение для группы, то есть оно может быть:

- константой;
- агрегатной функцией, которая оперирует всеми значениями аргумента в пределах группы и агрегирует их в одно значение (например, в сумму);
- выражением, идентичным стоящему во фразе GROUP BY;
- выражением, объединяющим приведенные выше варианты.

Самым простым вариантом использования фразы GROUP BY является группировка по значениям одного столбца.

Запрос: Количество клиентов по городам.

```
SELECT IdCity, COUNT(*) AS 'Кол-во клиентов'
FROM Customer
GROUP BY IdCity
```

Если в запросе используются фразы и WHERE, и GROUP BY, строки, не удовлетворяющие условию фразы WHERE, исключаются до выполнения группировки. Вследствие этого группировка производится только по тем строкам, которые удовлетворяют условию.

Запрос: Количество клиентов по городам с фамилией ‘Иванов’.

```
SELECT IdCity, COUNT(*) AS 'Кол-во клиентов'
FROM Customer
WHERE LName = 'Иванов'
GROUP BY IdCity
```

SQL позволяет группировать строки таблицы и по нескольким столбцам. В этом случае имена столбцов перечисляются во фразе GROUP BY через запятую.

Запрос: Количество клиентов по каждой фамилии и имени.

```
SELECT LName, FName, COUNT(*)
FROM Customer
```

```
GROUP BY LName, FName
```

Для отбора строк среди полученных групп применяется фраза HAVING. Она играет такую же роль для групп, что и фраза WHERE для исходных таблиц, и может использоваться лишь при наличии фразы GROUP BY. В предложении SELECT фразы WHERE, GROUP BY и HAVING обрабатываются в следующем порядке:

1. Фразой WHERE отбираются строки, удовлетворяющие указанному в ней условию;
2. Фраза GROUP BY группирует отобранные строки;
3. Фразой HAVING отбираются группы, удовлетворяющие указанному в ней условию.

Значение условия, указываемого во фразе HAVING, должно быть уникальным для всех строк каждой группы. Поэтому правила использования имен столбцов и агрегатных функций во фразе HAVING такие же, как и для фразы SELECT при наличии фразы GROUP BY. Это значит, что во фразе HAVING в качестве операндов сравнения можно использовать только группируемые столбцы или агрегатные функции.

Запрос: Список городов, количество клиентов из которых больше 10.

```
SELECT IdCity
FROM Customer
GROUP BY IdCity
HAVING COUNT(*)>10
```

Задание для самостоятельной работы: Сформулируйте на языке SQL запросы на выборку следующих данных:

– Список всех заказов с указанием их суммарной стоимости;
– Список клиентов, которые за заданный период (например, сентябрь 2010 года) совершили более 3 заказов.

Сложные (многотабличные запросы)

В SQL сложные запросы являются комбинацией простых SQL-запросов. Каждый простой запрос в качестве ответа возвращает набор записей (таблицу), а комбинация простых запросов возвращает результат тех или иных операций над ответами на простые запросы.

В SQL сложные запросы получаются из других запросов следующими способами:

– вложением SQL-выражения запроса в SQL-выражение другого запроса. Первый из них называют подзапросом, а второй — внешним или основным запросом;
– применением к SQL-запросам операторов объединения и соединения наборов записей, возвращаемых запросами. Эти операторы называют теоретико-множественными или реляционными.

Подзапросы

Подзапрос — это запрос на выборку данных, вложенный в другой запрос. Подзапрос всегда заключается в круглые скобки и выполняется до содержащего выражения. Внешний запрос, содержащий подзапрос, если только он сам не является подзапросом, не обязательно должен начинаться с оператора SELECT. В свою очередь,

подзапрос может содержать другой подзапрос и т. д. При этом сначала выполняется подзапрос, имеющий самый глубокий уровень вложения, затем содержащий его подзапрос и т. д. Часто, но не всегда, внешний запрос обращается к одной таблице, а подзапрос — к другой. На практике именно этот случай наиболее интересен.

Простые подзапросы

Простые подзапросы характеризуются тем, что они формально никак не связаны с содержащими их внешними запросами. Это обстоятельство позволяет сначала выполнить подзапрос, результат которого затем используется для выполнения внешнего запроса. Кроме простых подзапросов, существуют еще и связанные (коррелированные) подзапросы, которые будут рассмотрены в следующем разделе.

Рассматривая простые подзапросы, следует выделить три частных случая:

- подзапросы, возвращающие единственное значение;
- подзапросы, возвращающие список значений из одного столбца таблицы;
- подзапросы, возвращающие набор записей.

Тип возвращаемой подзапросом таблицы определяет, как можно ее использовать и какие операторы можно применять в содержащем выражении для взаимодействия с этой таблицей. По завершении выполнения содержащего выражения таблицы, возвращенные любым подзапросом, выгружаются из памяти. Таким образом, подзапрос действует как временная таблица, областью видимости которой является выражение (т. е. после завершения выполнения выражения сервер высвобождает всю память, отведенную под результаты подзапроса).

Подзапросы, возвращающие единственное значение

Допустим, из таблицы Customer требуется выбрать данные обо всех клиентах из Казани. Это можно сделать с помощью следующего запроса.

```
SELECT *
FROM Customer
WHERE IdCity = (SELECT idCity FROM City WHERE CityName = 'Казань')
```

В данном запросе сначала выполняется подзапрос (SELECT idCity FROM City WHERE CityName = 'Казань'). Он возвращает единственное значение (а не набор записей, поскольку по полю City организовано ограничение уникальности) – уникальный идентификатор города Казань. Если сказать точнее, то данный подзапрос возвращает единственную запись, содержащую единственное поле. Далее выполняется внешний запрос, который выводит все столбцы таблицы Customer и записи, в которых значение столбца IdCity равно значению, полученному с помощью подзапроса. Таким образом, сначала выполняется подзапрос, а затем внешний запрос, использующий результат подзапроса.

Задание для самостоятельной работы: По аналогии с предыдущим примером сформулируйте запрос, возвращающий все заказы, в которых содержится заданный товар (по названию товара).

Подзапросы, возвращающие список значений из одного столбца таблицы

Подзапрос, вообще говоря, может возвращать несколько записей. Чтобы в этом случае в условии внешнего оператора WHERE можно было использовать операторы сравнения, требующие единственного значения, используются кванторы, такие как ALL (все) и SOME (или ANY) (некоторый).

Рассмотрим общий случай использования запросов с кванторами ALL и SOME. Пусть имеются две таблицы: T1, содержащая как минимум столбец A, и T2, содержащая, по крайней мере, один столбец B. Тогда запрос с квантором ALL можно сформулировать следующим образом:

```
SELECT A FROM T1
WHERE A оператор_сравнения ALL (SELECT B FROM T2)
```

Здесь оператор_сравнения обозначает любой допустимый оператор сравнения. Данный запрос должен вернуть список всех тех значений столбца A, для которых оператор сравнения истинен для всех значений столбца B.

Запрос с квантором SOME, очевидно, имеет аналогичную структуру. Он должен вернуть список всех тех значений столбца A, для которых оператор сравнения истинен хотя бы для какого-нибудь одного значения столбца B.

Запрос: Список всех клиентов, проживающих в городах Казань или Елабуга.

```
SELECT *

FROM Customer

WHERE IdCity = SOME(SELECT IdCity FROM City WHERE CityName IN ('Казань', 'Елабуга'))
```

Предыдущий запрос может быть также реализован и с использованием оператора IN, который рассматривался в разделе “Фильтрация данных”.

```
SELECT *

FROM Customer

WHERE IdCity IN (SELECT IdCity FROM City WHERE CityName IN ('Казань', 'Елабуга'))
```

Напомним — он проверяет вхождение элемента во множество, в качестве элемента может выступать имя столбца или скалярное выражение, а в качестве множества — явно заданный список значений или подзапрос. Использование подзапроса в качестве второго операнда IN также как и кванторы позволяет избежать ограничения на единственность значения, возвращаемого подзапросом.

С помощью оператора IN можно проверять не только наличие значения в наборе значений, но и его отсутствие. Делается это добавлением оператора отрицания NOT. Вот другой вариант предыдущего запроса:

```
SELECT *

FROM Customer

WHERE IdCity NOT IN (SELECT IdCity FROM City WHERE CityName IN ('Казань', 'Елабуга'))
```

Этот запрос возвращает всех клиентов, кроме тех которые проживают в городах Казань и Елабуга.

Аналогичный запрос с использование квантора ALL:

```
SELECT *

FROM Customer

WHERE IdCity != ALL(SELECT IdCity FROM City WHERE CityName IN ('Казань', 'Елабуга'))
```

Задание для самостоятельной работы: Сформулируйте запрос, возвращающий список всех клиентов (с указанием фамилии и имени), совершивших заказ за определенный период времени.

Подзапросы, возвращающие набор записей

Подзапрос можно вставлять не только в операторы WHERE и HAVING, но и в оператор FROM.

```
SELECT t.столбец1, t.столбец2, ... , t.столбецn
FROM (SELECT ... ) t WHERE ...
```

Здесь таблице, возвращаемой подзапросом в операторе FROM, присваивается псевдоним t, а внешний запрос выделяет столбцы этой таблицы и, возможно, записи в соответствии с некоторым условием, которое указано в операторе WHERE.

Связанные (коррелированные) подзапросы

Все приведенные до сих пор запросы не зависели от своих содержащих выражений, т. е. могли выполняться самостоятельно и представлять свои результаты для проверки. Связанный подзапрос (коррелированный), напротив, зависит от содержащего выражения, из которого он ссылается на один или более столбцов. В отличие от несвязанного подзапроса, который выполняется непосредственно перед выполнением содержащего выражения, связанный подзапрос выполняется по разу для каждой строки-кандидата (это строки, которые предположительно могут быть включены в окончательные результаты). Например, следующий запрос использует связанный подзапрос для подсчета количества заказов у каждого клиента. Затем основной запрос выбирает тех клиентов, у которых больше одного заказа.

```
SELECT *
FROM Customer c
WHERE 1 < (SELECT COUNT(*) FROM [Order] r WHERE r.IdCust = c.IdCust)
```

Ссылка на c.idCust в самом конце подзапроса - это то, что делает этот подзапрос связанным. Чтобы подзапрос мог выполняться, основной запрос должен поставлять значения для c.IdCust. В данном случае основной запрос извлекает из таблицы Customer все строки и выполняет по одному подзапросу для всех клиентов, передавая в него соответствующий Id клиента при каждом выполнении. Если подзапрос возвращает значение большее одного, условие фильтрации выполняется и строка добавляется в результирующий набор.

Связанные подзапросы часто используются с условиями сравнения (в предыдущем примере <) и вхождения в диапазон, но самый распространенный оператор, применяемый в условиях со связанными подзапросами, - это оператор EXISTS (существует). Оператор EXISTS применяется, если требуется показать, что связь есть, а количество связей при этом не имеет значения. Например, следующий запрос возвращает список всех товаров, которые когда-либо заказывали.

```
SELECT IdProd, [Description]
FROM Product p
WHERE EXISTS (SELECT * FROM OrdItem oi WHERE oi.IdProd = p.IdProd)
```

При использовании оператора EXISTS подзапрос может возвращать ни одной, одну или много строк, а условие просто проверяет, возвращены ли в результате выполнения подзапроса строки (все равно сколько). Если взглянуть на блок SELECT подзапроса,

можно увидеть, что он состоит из единственного литерала *. Для условия основного запроса имеет значение только факт наличия возвращенных строк, а что именно было возвращено подзапросом - не важно. Поэтому подзапрос может возвращать все, что вам вздумается, но все же при использовании EXISTS принято задавать SELECT *.

Для поиска подзапросов, не возвращающих строки, можно использовать оператор EXISTS совместно с оператором отрицания NOT. В частности чтобы предыдущий запрос возвращал все товары, которые ни разу не заказывались, его можно модифицировать следующим образом.

```
SELECT IdProd, [Description]
FROM Product p
WHERE NOT EXISTS (SELECT * FROM OrdItem oi WHERE oi.IdProd = p.IdProd)
```

Задание для самостоятельной работы: Сформулируйте запрос, возвращающий список всех заказов с суммарной стоимость более заданной величины.

Операции соединения

При проектировании базы данных стремятся создавать таблицы, в каждой из которых содержалась бы информация об одном и только одном типе сущности. Это облегчает модификацию базы данных и поддержку ее целостности. Однако сущности могут быть взаимосвязанными. Клиенты связаны с городами по признаку проживания, заказы осуществляют клиенты, товары входят в состав заказов и т. д. Связь между таблицами устанавливается за счет размещения столбца первичного ключа одной таблицы, которая называется родительской, в другой взаимосвязанной таблице, которая называется дочерней. Столбец (или совокупность столбцов) дочерней таблицы, определенный для связи с родительской таблицей, называется внешним ключом. Так, например, таблица Customer содержит столбец IdCity, который для каждой строки клиента содержит значение первичного ключа того города, в котором проживает данный клиент. Наличие внешних ключей является основой для инициирования поиска по многим таблицам.

Одна из наиболее важных особенностей предложения SELECT — это способность использования связей между различными таблицами, а также вывода содержащейся в них информации. Операция, которая приводит к соединению из двух таблиц всех пар строк, для которых выполняется заданное условие, называется соединением таблиц.

Соединение таблиц во фразе WHERE по равенству значений столбцов различных таблиц

Соединение таблиц может быть указано во фразе WHERE или во фразе FROM. Сначала рассмотрим первый вариант. Большинство запросов, имеющих несколько таблиц во фразе FROM, содержат фразу WHERE, в которой указаны условия, попарно сравнивающие столбцы из различных таблиц. Такое условие называется условием соединения. В этом случае SQL предполагает сцепление только тех пар строк из разных таблиц, для которых условие соединения принимает истинное значение. Фраза WHERE помимо условия соединения может также содержать другие условия, каждое из которых ссылается на столбцы соединенной таблицы. Эти условия производят отбор строк соединенной таблицы.

Если таблицы соединяются по равенству значений пары столбцов (группы столбцов) из различных таблиц, такая операция называется соединением таблиц по равенству. Соединение по равенству позволяет соединить только те пары строк, которые действительно взаимосвязаны друг с другом. Так, например, мы можем соединить

таблицы городов и клиентов по условию City.IdCity = Customer.IdCity. В таком варианте мы соединяем таблицы осмысленно, так как каждая строка таблицы Customer соединяется только с одной строкой соответствующего города. На базе таблиц City и Customer мы получаем таблицу со столбцами из обеих таблиц, имеющую строки с понятным смыслом. Можно также сказать, что в таблицу Customer вместо столбца IdCity мы вставляем все характеристики (столбцы) соответствующего города из таблицы City. Соединение таблиц используется, когда необходимо вывести значения столбцов:

- разных таблиц;
- одной таблицы, но отвечающих условию, заданному на другой таблице.

Эти два варианта, а также их комбинация, характерны для любого вида соединения, а не только по равенству. Рассмотрим следующие примеры.

```
SELECT FName, LName, CityName
FROM Customer, City
WHERE Customer.IdCity = City.IdCity
```

Этот запрос возвращает список всех клиентов с указанием названий городов, в которых они проживают. Этот вид запросов характерен тем, что фраза WHERE содержит только условие соединения, а список фразы SELECT содержит имена столбцов из различных таблиц.

До тех пор, пока запрос относится к одной таблице, обращение к столбцам по их именам не вызывает проблем - в таблице все имена столбцов должны быть неповторяющимися. Однако как только запрос соединяет несколько таблиц, может возникнуть неоднозначность при ссылках на столбцы с одинаковыми именами из разных таблиц. Для разрешения этой неоднозначности во фразах SELECT и WHERE (как и в некоторых других фразах) имена столбцов необходимо уточнять именами таблиц. Запишем предыдущий запрос с полным уточнением имен.

```
SELECT Customer.FName, Customer.LName, City.CityName
FROM Customer, City
WHERE Customer.IdCity = City.IdCity
```

В этом запросе мы уточнили имена столбцов во фразах SELECT и WHERE, хотя в предложение SELECT это было не обязательно, так как используются неповторяющиеся имена. Тем не менее, рекомендуется при соединении таблиц для наглядности уточнять имена столбцов. Однако на практике для задания более лаконичных имен часто используют короткие синонимы таблиц, по которым можно сослаться на них в любых других местах запроса. Синоним указывается сразу после имени таблицы в предложении FROM. В частности предыдущий запрос с использование синонимов для таблиц можно записать более компактным образом.

```
SELECT k.FName, k.LName, c.CityName
FROM Customer k, City c
WHERE k.IdCity = c.IdCity
```

Следующий запрос отбирает всех клиентов из Казани с фамилией Иванов

```
SELECT K.IdCust, k.FName
FROM Customer k, City c
WHERE k.IdCity = c.IdCity AND k.LName = 'Иванов' AND c.CityName = 'Казань'
```

В этом запросе помимо условия соединения используется также отбор строк по условиям, заданным для разных таблиц.

SQL позволяет формулировать запросы, которые предполагают использование трех и более таблиц. При этом следует применять ту же методику соединения, что и для двух таблиц. Рассмотрим простой пример соединения трех таблиц.

Запрос: Список всех клиентов, которые когда-либо заказывали товар с кодом 1.

```
SELECT DISTINCT c.IdCust, c.FName, c.LName
FROM Customer c, [Order] o, OrdItem oi
WHERE c.IdCust = o.IdCust AND o.IdOrd = oi.IdOrd AND oi.IdProd = 1
```

Сформулируем общую процедуру составления многотабличного запроса.

1. Определить множество таблиц, необходимых для ответа на запрос. В это множество должны входить таблицы, на столбцах которых сформулированы условия, а также те, столбцы которых необходимо вывести. Это так называемые базовые таблицы запроса.

2. В структуре взаимосвязанных таблиц найти путь, соединяющий базовые таблицы. Это так называемый путь вычисления запроса. В результате вы получите перечень таблиц, необходимых для формулировки запроса. Это так называемые таблицы запроса.

3. Во фразе FROM перечислить необходимые таблицы.

4. Во фразе WHERE соединить таблицы запроса и при необходимости задать условия отбора строк в базовых таблицах запроса.

5. Во фразе SELECT перечислить выводимые столбцы.

Задание для самостоятельной работы: Сформулируйте запрос, возвращающий список товаров в заданном заказе (по заданному IdOrd). Результат должен включать следующие поля: название товара, цена, количество, стоимость.

Соединения с использованием фразы FROM

Все рассмотренные выше типы и способы соединения таблиц можно (и рекомендуется, поскольку соединения во фразе WHERE считаются устаревшими) осуществлять и с помощью фразы FROM. В ней, в соответствии со стандартом SQL, можно не только перечислить имена таблиц, участвующих в запросе, но и указать их соединение, используя следующий синтаксис.

таблица [INNER | {FULL | LEFT | RIGHT} [OUTER]] JOIN таблица {ON условие}

Внутреннее соединение

В операторе JOIN внутреннее соединение указывается ключевым словом INNER (впрочем, его можно опустить, так как соединение двух таблиц является внутренним по умолчанию). Условие соединения указывается после ключевого слова ON. В этом случае внутреннее соединение с помощью фразы FROM JOIN очень похоже на соединение с использованием фразы WHERE. Запишем первый пример с предыдущего раздела с использование оператора JOIN.

Запрос: Список всех клиентов с указанием названий городов, в которых они проживают

```
SELECT FName, LName, CityName
FROM Customer k JOIN
    City c ON k.IdCity = c.IdCity
```

При соединении с использованием фразы FROM дополнительное условие можно для увеличения наглядности запроса помещать во фразу WHERE. В этом случае второй пример с предыдущего раздела примет такой вид.

Запрос: Список всех клиентов из Казани с фамилией Иванов

```
SELECT K.IdCust, k.FName
FROM Customer k INNER JOIN
    City c ON k.IdCity = c.IdCity
WHERE k.LName = 'Иванов' AND c.CityName = 'Казань'
```

Внешнее соединение

Все соединения таблиц, рассмотренные до сих пор, являются внутренними. Во всех примерах вместо ключевого слова JOIN можно писать INNER JOIN (внутреннее соединение). Из таблицы, получаемой при внутреннем соединении, отбраковываются все записи, для которых нет соответствующих записей одновременно в обеих соединяемых таблицах. При внешнем соединении такие несоответствующие записи сохраняются. В этом и заключается отличие внешнего соединения от внутреннего.

С помощью специальных ключевых слов LEFT OUTER, RIGHT OUTER и FULL OUTER, написанных перед JOIN, можно выполнить соответственно левое, правое и полное соединение. В SQL-выражении запроса таблица, указанная слева от оператора JOIN, называется левой, а указанная справа от него — правой.

При левом внешнем соединении несоответствующие записи, имеющиеся в левой таблице, сохраняются в результатной таблице, а имеющиеся в правой — удаляются. Значения столбцов из правой таблицы во всех строках, не имеющих соответствия с левой таблицей, принимают значение NULL.

При правом внешнем соединении несоответствующие записи, имеющиеся в правой таблице, сохраняются в результатной таблице, а имеющиеся в левой — удаляются. Значения столбцов из левой таблицы во всех строках, не имеющих соответствия с правой таблицей, принимают значение NULL.

Соответственно левое и правое внешние соединения различаются только порядком следования таблиц.

При полном внешнем соединении двух таблиц результирующая таблица содержит все строки внутреннего соединения этих таблиц, а также не включенные им строки и первой, и второй таблиц (дополненные значениями NULL для отсутствующих столбцов).

В следующем примере возвращается полный список городов с указанием количества клиентов из каждого из них

```
SELECT c.CityName, a.CountCity
FROM City c LEFT OUTER JOIN
 (SELECT IdCity, COUNT(*) AS CountCity
  FROM Customer
  GROUP BY IdCity) a ON c.IdCity = a.IdCity
```

ORDER BY c.CityName

Если в данном запросе заменить левое внешнее соединение на внутреннее, то из результата будут потеряны города, из которых нет ни одного клиента (проверьте это заменив LEFT OUTER JOIN на INNER JOIN и объясните причину разницы). Обратите внимание, что таблица City соединяется не с таблицей, а с подзапросом, которому задан псевдоним a.

Задание для самостоятельной работы: Сформулируйте на языке SQL запросы на выборку следующих данных (с использование оператора JOIN для соединения таблиц):

- список всех товаров, которые когда-либо заказывал заданный клиент;
- список всех клиентов, не имеющих ни одного заказа.

Множественные операции

Множественные операции выполняются над наборами записей (таблицами), полученными в результате запросов, и, в свою очередь, возвращают таблицу.

В стандарте SQL множественные операции имеют следующий синтаксис:

запрос {UNION | INTERSECT | EXCEPT} [DISTINCT | ALL] запрос

где запрос является предложением SELECT. Отличаются эти операции тем, какие строки возвращенных запросами таблиц отбираются в новую результирующую таблицу:

- UNION — все строки таблиц, возвращенных обоими запросами;
- INTERSECT — только те строки, которые имеются в таблицах обоих запросов;
- EXCEPT — только те строки таблицы первого запроса, которых нет среди строк таблицы второго запроса.

Запросы, содержащие множественные операторы, называются составными.

Стандарт SQL определяет следующие правила относительно повторяющихся строк в таблицах:

– базовые таблицы не могут содержать повторяющихся строк (это принципиальное требование реляционной модели данных);
– результирующие таблицы запросов могут содержать повторяющиеся строки, если это не запрещено ключевым словом DISTINCT во фразе SELECT;
– результирующие таблицы множественных операций не могут содержать повторяющихся строк, если это не разрешено ключевым словом ALL.

Таким образом, именно ключевые слова ALL и DISTINCT указывают, допускаются ли в результирующей таблице повторяющиеся строки. В запросах при отсутствии явного указания предполагается использование ключевого слова ALL, а во множественных операциях — DISTINCT.

Таблицы, используемые в качестве операндов множественной операции, должны быть совместимы. Под этим подразумевается следующее:

- обе таблицы должны иметь одинаковое количество столбцов;
- соответствующие пары столбцов должны быть одинаковых или совместимых типов.

Объединение наборов записей

Нередко требуется объединить записи двух или более таблиц с похожими структурами в одну таблицу. Иначе говоря, к набору записей, возвращаемому одним

запросом, требуется добавить записи, возвращаемые другим запросом. Для этого служит оператор UNION (объединение):

Запрос1 UNION Запрос2;

При этом в результатной таблице остаются только отличающиеся записи. Чтобы сохранить в ней все записи, после оператора UNION следует написать ключевое слово ALL.

К базе данных Sales сложно сформулировать осмысленный запрос с объединением, который бы имел какую-либо практическую ценность. Поэтому в качестве примера рассмотрим объединение результатов выполнения запросов, возвращающих просто константные значения.

Запрос: Объединение с исключением дублирующих строк

```
SELECT 1, 'Один'
UNION
SELECT 1, 'Один'
UNION
SELECT 2, 'Два'
```

Запрос: Объединение с сохранением дубликатов

```
SELECT 1, 'Один'
UNION ALL
SELECT 1, 'Один'
UNION ALL
SELECT 2, 'Два'
```

Пересечение наборов записей

Пересечение двух наборов записей осуществляется с помощью оператора INTERSECT (пересечение), возвращающего таблицу, записи в которой содержатся одновременно в двух наборах:

Запрос1 INTERSECT Запрос2;

```
(SELECT 1, 'Один'
 UNION
 SELECT 2, 'Два'
 UNION
 SELECT 3, 'Три')

INTERSECT
```

```
(SELECT 1, 'Один'
 UNION
 SELECT 2, 'Два'
 UNION
 SELECT 4, 'Четыре')
```

Разность наборов записей

Для получения записей, содержащихся в одном наборе и отсутствующих в другом, служит оператор EXCEPT(за исключением):

Запрос1 EXCEPT Запрос2;

```
(SELECT 1, 'Один'
 UNION
 SELECT 2, 'Два'
 UNION
 SELECT 3, 'Три')

EXCEPT

(SELECT 1, 'Один'
 UNION
 SELECT 2, 'Два'
 UNION
 SELECT 4, 'Четыре')
```

Запросы на добавление, изменение и удаление данных в таблицах

Запросы, рассмотренные ранее, были направлены на то, чтобы получить данные, содержащиеся в существующих таблицах базы данных. Главным ключевым словом таких запросов на выборку данных является SELECT. Запросы на выборку данных всегда возвращают виртуальную таблицу, которая отсутствует в базе данных и создается временно лишь для того, чтобы представить выбранные данные пользователю. При создании и дальнейшем сопровождении базы данных обычно возникает задача добавления новых и удаления ненужных записей, а также изменения содержимого ячеек таблицы. В SQL для этого предусмотрены операторы INSERT (вставить), DELETE (удалить) и UPDATE (изменить). Запросы, начинающиеся с этих ключевых слов, не возвращают данные в виде виртуальной таблицы, а изменяют содержимое уже существующих таблиц базы данных. Запросы на модификацию (добавление, удаление и изменение) данных могут содержать вложенные запросы на выборку данных из той же самой таблицы или из других таблиц, однако сами не могут быть вложены в другие запросы. Таким образом, операторы INSERT, DELETE и UPDATE в SQL-выражении могут находиться только в самом начале.

Добавление новых записей

Для вставки записей в таблицу используется оператор INSERT, который имеет несколько форм:

– INSERT INTO имяТаблицы VALUES (списокЗначений)

вставляет запись в указанную таблицу и заполняет эту запись значениями из списка, указанного за ключевым словом VALUES. При этом первое в списке значение вводится в первый столбец таблицы, второе значение — во второй столбец и т. д. Порядок столбцов задается при создании таблицы. Данная форма оператора INSERT не очень надежна, поскольку нетрудно ошибиться в порядке вводимых значений. Более надежной и гибкой является следующая форма.

– INSERT INTO имяТаблицы (списокСтолбцов) VALUES (списокЗначений)

вставляет запись в указанную таблицу и вводит в заданные столбцы значения из указанного списка. При этом в первый столбец из списокСтолбцов вводится первое значение из списокЗначений, во второй столбец — второе значение и т. д. Порядок имен столбцов в списке может отличаться от их порядка, заданного при создании таблицы. Столбцы, которые не указаны в списке, заполняются значением NULL. Рекомендуется использовать именно данную форму оператора INSERT. Следующий запрос добавляет новую запись в справочник городов.

```
INSERT INTO City(CityName)
VALUES('Калуга')
```

Обратите внимание, что столбец IdCity не задается, поскольку он является счетчиком и заполняется СУБД автоматически.

– INSERT INTO имяТаблицы (списокСтолбцов) SELECT ...

вставляет в указанную таблицу записи, возвращаемые запросом на выборку. На практике нередко требуется загрузить в одну таблицу данные из другой таблицы. Например, следующий запрос вставляет в таблицу City сразу два города, возвращаемых запросом с объединением.

```
INSERT INTO City(CityName)
```

```
SELECT 'Уфа'
UNION
SELECT 'Волгоград'
```

Удаление записей

Для удаления записей из таблицы применяется оператор DELETE:

```
DELETE FROM имяТаблицы WHERE условие;
```

Данный оператор удаляет из указанной таблицы записи (а не отдельные значения столбцов), которые удовлетворяют указанному условию. Условие — это логическое выражение, различные конструкции которого были рассмотрены в предыдущих лабораторных занятиях.

Следующий запрос удаляет записи из таблицы Customer, в которой значение столбца LName равно 'Иванов':

```
DELETE FROM Customer
WHERE LName = 'Иванов'
```

Если таблица содержатся сведения о нескольких клиентах с фамилией Иванов, то все они будут удалены.

В операторе WHERE может находиться подзапрос на выборку данных (оператор SELECT). Подзапросы в операторе DELETE работают точно так же, как и в операторе SELECT. Следующий запрос удаляет всех клиентов из города Москва, при этом уникальный идентификатор города возвращается с помощью подзапроса.

```
DELETE FROM Customer
WHERE IdCity IN (SELECT IdCity FROM City WHERE CityName = 'Москва')
```

Transact-SQL расширяет стандартный SQL, позволяя использовать в инструкции DELETE еще одно предложение FROM. Это расширение, в котором задается соединение, может быть использовано вместо вложенного запроса в предложении WHERE для указания удаляемых строк. Оно позволяет задавать данные из второго FROM и удалять соответствующие строки из таблицы в первом предложении FROM. В частности предыдущий запрос может быть переписан следующим образом

```
DELETE FROM Customer
FROM Customer k INNER JOIN
    City c ON k.IdCity = c.IdCity AND c.CityName = 'Москва'
```

Операция удаления записей из таблицы является опасной в том смысле, что связана с риском необратимых потерь данных в случае семантических (но не синтаксических) ошибок при формулировке SQL-выражения. Чтобы избежать неприятностей, перед удалением записей рекомендуется сначала выполнить соответствующий запрос на выборку, чтобы просмотреть, какие записи будут удалены. Так, например, перед выполнением рассмотренного ранее запроса на удаление не помешает выполнить соответствующий запрос на выборку.

```
SELECT *
FROM Customer k INNER JOIN
    City c ON k.IdCity = c.IdCity AND c.CityName = 'Москва'
```

Для удаления всех записей из таблицы достаточно использовать оператор DELETE без ключевого слова WHERE. При этом сама таблица со всеми определенными в ней

столбцами сохраняется и готова для вставки новых записей. Например, следующий запрос удаляет записи обо всех товарах.

```
DELETE FROM Product
```

Задание для самостоятельной работы: Сформулируйте на языке SQL запрос на удаление всех заказов, не имеющих в составе ни одного товара (т. е. все пустые заказы).

Обновление данных

Для изменения значений столбцов таблицы применяется оператор UPDATE (изменить, обновить). Чтобы изменить значения в одном столбце таблицы в тех записях, которые удовлетворяют некоторому условию, следует выполнить такой запрос.

```
UPDATE имяТаблицы
SET имяСтолбца = значение
WHERE условие;
```

За ключевым словом SET (установить) следует выражение равенства, в левой части которого указывается имя столбца, а в правой — выражение, значение которого следует сделать значением данного столбца. Эти установки будут выполнены в тех записях, которые удовлетворяют условию в операторе WHERE.

Чтобы одним оператором UPDATE установить новые значения сразу для нескольких столбцов, вслед за ключевым словом SET записываются соответствующие выражения равенства, разделенные запятыми.

```
UPDATE имяТаблицы
SET имяСтолбца1 = значение1, имяСтолбца2 = значение2, ... , имяСтолбцаN = значениеN
WHERE условие;
```

Например, следующий запрос изменяет фамилию и имя клиента с кодом 5.

```
UPDATE Customer
SET FName='Иван', LName='Иванов'
WHERE IdCust = 5
```

Использование оператора WHERE в инструкции UPDATE не обязательно. Если он отсутствует, то указанные в SET изменения будут произведены для всех записей таблицы.

Так же как и в инструкции DELETE условие в операторе WHERE инструкции UPDATE может содержать подзапросы, в том числе и связанные.

Transact-SQL расширяет стандартный SQL, позволяя использовать в инструкции UPDATE предложение FROM (по аналогии с DELETE). Это расширение, в котором задается соединение, может быть использовано вместо вложенного запроса в предложении WHERE для указания обновляемых строк.

Если обновляемый объект тот же самый, что и объект в предложении FROM, и в предложении FROM имеется только одна ссылка на этот объект, псевдоним объекта указывать необязательно. Если обновляемый объект встречается в предложении FROM несколько раз, одна и только одна ссылка на этот объект не должна указывать псевдоним таблицы. Все остальные ссылки на объект в предложении FROM должны включать псевдоним объекта.

Предположим, что требуется сделать 5% скидку по тем заказам клиентов, суммарная стоимость которых превышает 1000. Для этого следует изменить значения столбца Price, просто умножить их на 0,95. Однако эти изменения должны быть

выполнены, только если суммарная стоимость заказа превышает 1000. Таким образом, в качестве критерия обновления записей в таблице OrdItem может быть задан запрос возвращающий список всех заказов с суммарной стоимостью более 1000.

```
UPDATE OrdItem
SET Price = Price * 0.95
FROM OrdItem o INNER JOIN
 (SELECT IdOrd
  FROM OrdItem
  GROUP BY IdOrd
  HAVING SUM(Qty*Price) > 1000) a ON o.IdOrd = a.IdOrd
```

Операция изменения записей, как и их удаление, связана с риском необратимых потерь данных в случае семантических ошибок при формулировке SQL-выражения. Например, стоит только забыть написать оператор WHERE, и будут обновлены значения во всех записях таблицы. Чтобы избежать подобных неприятностей, перед обновлением записей рекомендуется выполнить соответствующий запрос на выборку, чтобы просмотреть, какие записи будут изменены. Например, перед выполнением приведенного ранее запроса на обновление данных не помешает выполнить соответствующий запрос на выборку данных.

```
SELECT *
FROM OrdItem o INNER JOIN
 (SELECT IdOrd
  FROM OrdItem
  GROUP BY IdOrd
  HAVING SUM(Qty*Price) > 1000) a ON o.IdOrd = a.IdOrd
```

Задание для самостоятельной работы: Сформулируйте на языке SQL запрос имитирующий поступление на склад новой партий определенного товара (Обновление столбца InStock в таблице Product).

2.4.2. Применение переменных и управляющих конструкций в SQL-операторах.

Программирование на T-SQL

Синтаксис и соглашения T-SQL

Правила формирования идентификаторов

Все объекты в SQL Server имеют имена (идентификаторы). Примерами объектов являются таблицы, представления, хранимые процедуры и т.д. Идентификаторы могут включать до 128 символов, в частности, буквы, символы _ @ $ # и цифры. Первый символ всегда должен быть буквенным. Для переменных и временных таблиц используются специальные схемы именования. Имя объекта не может содержать пробелов и совпадать с зарезервированным ключевым словом SQL Server, независимо от используемого регистра символов. Путем заключения идентификаторов в квадратные скобки, в именах объектов можно использовать запрещенные символы.

Завершение инструкции

Стандарт ANSI SQL требует помещения в конце каждой инструкции точки с запятой. В то же время при программировании на языке T-SQL точка с запятой не обязательна.

Комментарии

Язык T-SQL допускает использование комментариев двух стилей: ANCI и языка C. Первый из них начинается с двух дефисов и заканчивается в конце строки:

```
-- Это однострочный комментарий стиля ANSI
```

Также комментарии стиля ANSI могут вставляться в конце строки инструкции:

```
SELECT CityName – извлекаемые столбцы
FROM City – исходная таблица
WHERE IdCity = 1; -- ограничение на строки
```

Редактор SQL может применять и удалять комментарии во всех выделенных строках. Для этого нужно выбрать соответствующие команды в меню *Правка* или на панели инструментов  .

Комментарии стиля языка C начинаются с косой черты и звездочки (/*) и заканчиваются теми же символами в обратной последовательности. Этот тип комментариев лучше использовать для комментирования блоков строк, таких как заголовки или большие тестовые запросы.

```
/*
Пример
многострочного
комментария
*/
```

Одним из главных достоинств комментариев стиля C является то, что многострочные запросы в них можно выполнять, даже не раскомментируя.

Пакеты T-SQL

Запросом называют одну инструкцию T-SQL, а пакетом — их набор. Вся последовательность инструкций пакета отправляется серверу из клиентских приложений как одна цельная единица.

SQL Server рассматривает весь пакет как рабочую единицу. Наличие ошибки хотя бы в одной инструкции приведет к невозможности выполнения всего пакета. В то же время грамматический разбор не проверяет имена объектов и схем, так как сама схема может измениться в процессе выполнения инструкции.

Файл сценария SQL и окно анализатора запросов (Query Analyzer) может содержать несколько пакетов. В данном случае все пакеты разделяют ключевые слова терминаторов. По умолчанию этим ключевым словом является GO, и оно должно быть единственным в строке. Все другие символы (даже комментарии) нейтрализуют разделитель пакета.

Отладка T-SQL

Когда редактор SQL обнаруживает ошибку, он отображает ее характер и номер строки в пакете. Дважды щелкнув на ошибке, можно сразу же переместиться к соответствующей строке.

В утилиту Management Studio версии SQL Server 2005 не включен отладчик языка T-SQL, — он присутствует в пакете Visual Studio.

SQL Server предлагает несколько команд, облегчающих отладку пакетов. В частности, команда PRINT отправляет сообщение без генерации результирующего набора данных. Команду PRINT можно использовать для отслеживания хода выполнения пакета. Когда анализатор запросов находится в режиме сетки, выполните следующий пакет:

```
SELECT CityName
FROM City
WHERE IdCity = 1;


PRINT 'Контрольная точка';
```

Результирующий набор данных отобразится в сетке и будет состоять из одной строки. В то же время во вкладке «Сообщения» отобразится следующий результат:

(строк обработано: 1)

Контрольная точка

Переменные

Переменные T-SQL создаются с помощью команды DECLARE, имеющей следующий синтаксис:

DECLARE @Имя_Переменной Тип_Данных [,

@Имя_Переменной Тип_Данных, …]

Все имена локальных переменных должны начинаться символом @. Например, для объявления локальной переменной UStr, которая хранит до 16 символов Unicode, можно использовать следующую инструкцию:

```
DECLARE @UStr varchar(16)
```

Используемые для переменных типы данных в точности совпадают с существующими в таблицах. В одной команде DECLARE через запятую может быть перечислено несколько переменных. В частности в следующем примере создаются две целочисленные переменные a и b:

```
DECLARE
  @a int,
  @b int
```

Область определения переменных (т.е. срок их жизни) распространяется только на текущий пакет. По умолчанию только что созданные переменные содержат пустые значения NULL и до включения в выражения должны быть инициализированы.

Задание значений переменных

В настоящее время в языке SQL предусмотрены два способа задания значения переменной — для этой цели можно использовать оператор SELECT или SET. С точки зрения выполняемых функций эти операторы действуют почти одинаково, не считая того, что оператор SELECT позволяет получить исходное присваиваемое значение из таблицы, указанной в операторе SELECT.

Оператор SET обычно используется для задания значений переменных в такой форме, какая более часто встречается в процедурных языках. В качестве типичных примеров применения этого оператора можно указать следующие:

```
SET @a = 1;
SET @b = @a * 1.5
```

Обратите внимание на то, что во всех этих операторах непосредственно осуществляются операции присваивания, в которых используются либо явно заданные значения, либо другие переменные. С помощью оператора SET невозможно присвоить переменной значение, полученное с помощью запроса; запрос должен быть выполнен отдельно и только после этого полученный результат может быть присвоен с помощью оператора SET. Например, попытка выполнения такого оператора вызывает ошибку:

```
DECLARE @c int
SET @c = COUNT(*) FROM City
SELECT @c
```

а следующий оператор выполняется вполне успешно:

```
DECLARE @c int
SET @c = (SELECT COUNT(*) FROM City)
SELECT @c
```

Оператор SELECT обычно используется для присваивания значений переменным, если источником информации, которая должна быть сохранена в переменной, является запрос. Например, действия, осуществляемые в приведенном выше коде, гораздо чаще реализуются с помощью оператора SELECT:

```
DECLARE @c int
SELECT @c = COUNT(*) FROM City
SELECT @c
```

Обратите внимание на то, что данный код немного понятнее (в частности, он более лаконичен, хотя и выполняет те же действия).

Таким образом, можно, сформулировать следующее общепринятое соглашение по использованию того и другого оператора.

- Оператор SET используется, если должна быть выполнена простая операция присваивания значения переменной, т.е. если присваиваемое значение уже задано явно в форме определенного значения или в виде какой-то другой переменной.
- Оператор SELECT применяется, если присваивание значения переменной должно быть основано на запросе.

Использование переменных в запросах SQL

Одним из полезных свойств языка T-SQL является то, что переменные могут использоваться в запросах без необходимости создания сложных динамических строк, встраивающих переменные в программный код. Динамический SQL продолжает свое существование, но одиночное значение можно изменить проще — с помощью переменной.

Везде, где в запросе может использоваться выражение, может использоваться и переменная. В следующем примере продемонстрировано использование переменной в предложении WHERE:

```
DECLARE @IdProd int;

SET @IdProd = 1;

SELECT [Description]

FROM Product

WHERE IdProd = @IdProd;
```

Глобальные системные переменные

В SQL Server имеется более тридцати глобальных переменных, не имеющих параметров, которые определяются и поддерживаются системой. Все глобальные переменные имеют префикс в виде двух символов @. Вы можете извлечь значение любой из них с помощью простого запроса SELECT, как в следующем примере:

```
SELECT @@CONNECTIONS
```

Здесь используется глобальная переменная @@CONNECTIONS для извлечения количества подключений к SQL Server со времени запуска программы.

Среди наиболее часто применяемых системных переменных можно отметить следующие:

- @@ERROR - Содержит номер ошибки, возникшей при выполнении последнего оператора T-SQL в текущем соединении. Если ошибка не обнаружена, содержит 0. Значение этой системной переменной переустанавливается после выполнения каждого очередного оператора. Если требуется сохранить содержащееся в ней значение, то это значение следует переносить в локальную переменную сразу же после выполнения оператора, для которого должен быть сохранен код ошибки.
- @@IDENTITY - Содержит последнее идентификационное значение, вставленное в базу данных в результате выполнения последнего оператора INSERT. Если в последнем операторе INSERT не произошла выработка идентификационного значения, системная переменная @@IDENTITY содержит NULL. Это утверждение остается справедливым, даже если отсутствие идентификационного значения было вызвано аварийным завершением при выполнении оператора. А если с помощью одного оператора осуществляется несколько операций вставки, этой системной переменной присваивается только последнее идентификационное значение.
- @@ROWCOUNT - Одна из наиболее широко используемых системных переменных. Возвращает информацию о количестве строк, затронутых последним оператором. Обычно применяется для контроля ошибок, отличных от тех, которые относятся к категории ошибок этапа прогона программы. Например, если в программе

обнаруживается, что после вызова на выполнение оператора DELETE с конструкцией WHERE количество затронутых строк равно нулю, то можно сделать вывод, что произошло нечто непредвиденное. После этого сообщение об ошибке может быть активизировано вручную.

! Следует отметить, что с версии SQL Server 2000 глобальные переменные принято называть функциями. Название глобальные сбивало пользователей с толку, позволяя думать, что область действия таких переменных шире, чем у локальных. Глобальным переменным часто ошибочно приписывалась возможность хранить информацию, независимо от того, включена она в пакет либо нет, что, естественно, не соответствовало действительности.

Средства управления потоком команд. Программные конструкции

В языке T-SQL предусмотрена большая часть классических процедурных средств управления ходом выполнения программы, в т.ч. условная конструкция и циклы.

Оператор IF. . .ELSE

Операторы IF. . .ELSE действуют в языке T-SQL в основном так же, как и в любых других языках программирования. Общий синтаксис этого оператора имеет следующий вид:

IF Логическое выражение
SQL инструкция I BEGIN Блок SQL инструкций END
[ELSE
SQL инструкция | BEGIN Блок SQL инструкций END]

В качестве логического выражения может быть задано практически любое выражение, результат вычисления которого приводит к возврату булева значения.

Следует учитывать, что выполняемым по условию считается только тот оператор, который непосредственно следует за оператором IF (ближайшим к нему). Вместо одного оператора можно предусмотреть выполнение по условию нескольких операторов, объединив их в блок кода с помощью конструкции BEGIN…END.

В приведенном ниже примере условие IF не выполняется, что предотвращает выполнение следующего за ним оператора.

```
IF 1 = 0
        PRINT 'Первая строка'
PRINT 'Вторая строка'
```

Необязательная команда ELSE позволяет задать инструкцию, которая будет выполнена в случае, если условие IF не будет выполнено. Подобно IF, оператор ELSE управляет только непосредственно следующей за ним командой или блоком кода заключенным между BEGIN…END.

Несмотря на то, что оператор IF выглядит ограниченным, его предложение условия может включать в себя мощные функции, подобно предложению WHERE. В частности это выражения IF EXISTS().

Выражение IF EXISTS() использует в качестве условия наличие какой-либо строки, возвращенной инструкцией SELECT. Так как ищутся любые строки, список столбцов в инструкции SELECT можно заменить звездочкой. Этот метод работает быстрее, чем проверка условия @@ROWCOUNT>0, потому что не требуется подсчет общего количества строк. Как только хотя бы одна строка удовлетворяет условию IF EXISTS(), запрос может продолжать выполнение.

В следующем примере выражение IF EXISTS используется для проверки наличия у клиента с кодом 1 каких-либо заказов перед удалением его из базы. Если по данному клиенту есть информация хотя бы по одному заказу, удаление не производится.

```
IF EXISTS(SELECT * FROM [Order] WHERE IdCust = 1)
        PRINT 'Невозможно удалить клиента поскольку в базе имеются связанные с ним записи'
ELSE
        BEGIN
                DELETE Customer
                WHERE IdCust = 1
                PRINT 'Удаление произведено успешно'
        END
```

Операторы WHILE, BREAK и CONTINUE

Оператор WHILE в языке SQL действует во многом так же, как и в других языках, с которыми обычно приходится работать программисту. По сути, в этом операторе до начала каждого прохода по циклу проверяется некоторое условие. Если перед очередным проходом по циклу проверка условия приводит к получению значения TRUE, осуществляется проход по циклу, в противном случае выполнение оператора завершается.

Оператор WHILE имеет следующий синтаксис:

```
WHILE Логическое выражение
SQL инструкция I
[BEGIN
   [BREAK]
   Блок SQL инструкций
   [CONTINUE]
END]
```

Безусловно, с помощью оператора WHILE можно обеспечить выполнение в цикле только одного оператора (по аналогии с тем, как обычно используется оператор IF), но на практике конструкции WHILE, за которыми не следует блок BEGIN. . .END, соответствующий полному формату оператора, встречаются редко.

Оператор BREAK позволяет немедленно выйти из цикла, не ожидая того, как будет выполнен проход до конца цикла и произойдет повторная проверка условного выражения.

Оператор CONTINUE позволяет прервать отдельную итерацию цикла. Кратко можно описать действие оператора CONTINUE так, что он обеспечивает переход в начало цикла WHILE. Сразу после обнаружения оператора CONTINUE в цикле, независимо от того, где он находится, происходит переход в начало цикла и повторное вычисление условного выражения (а если значение этого выражения больше не равно TRUE, осуществляется выход из цикла).

Следующий короткий сценарий демонстрирует использование оператора WHILE для создания цикла:

```
DECLARE @Temp int;
SET @Temp = 0;
WHILE @Temp < 3
        BEGIN
```

```
        PRINT @Temp;
        SET @Temp = @Temp + 1;
    END
```

Здесь в цикле целочисленная переменная @Temp увеличивается с 0 до 3 и на каждой итерации ее значение выводится на экран.

Оператор RETURN

Оператор RETURN используется для останова выполнения пакета, а следовательно, хранимой процедуры и триггера (рассматриваются в следующих лабораторных занятиях).